3) $T(n) = 2T(n/2 + 1/2) + n \Rightarrow 2T(n/2) + n$

soln $\Rightarrow$ $a = 2, b = 2,$ & $f(n) = n$, $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

compare $f(n) = n^{\log_b a} = n$ so $T(n) = O(n^{\log_b a} \log n)$

$T(n) = O(n \log n)$

Recursion method $\Rightarrow$ Backtracking method $\Rightarrow$

$T(n) = 2T(n/2) + n$

steps 1) understand the problem
- a) problem is divided into 2 subproblems
- b) each subproblem size $= n/2$
- c) extra work done at each level $= n$

step 2) $\Rightarrow$ level 0 (Root) $\Rightarrow$ i) work done $= n$

level 1 $\Rightarrow$ 2 subproblems $n/2$

work at this level $\Rightarrow n/2 + n/2 = n$

level 2 $\Rightarrow$ 4 subproblems $n/4$

work at this level $\Rightarrow n/4 + n/4 + n/4 + n/4 = n$

General level i $\Rightarrow$ a) no. of nodes $= 2^i$

b) work per node $= n/2^i$

c) Total work at level i; $2^i \times \frac{n}{2^i} = n$

step 3) find the heigh of the tree $\Rightarrow$

Recursion stops when $\Rightarrow$ $n/2^k = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$

step 4) Add work of All levels
- i) work per level $= n$
- ii) Number of levels $= \log n$

Total work $= n \times \log n$

step 5) final answer : $T(n) = O(n \log n)$

Notation $\Rightarrow$ i) Notation is a symbol or mathematical way to represent information
- ii) In algorithms, notation is used to represent time and space complexity
- iii) It helps us compare algorithm performance.

why notation use $\Rightarrow$ i) To measure algorithm efficiency
- ii) To compare two or more algorithms
- iii) to describe performance independent of machine and language
- iv) To analyze behavior for large input size (n)

Types of notation $\Rightarrow$ 1) Asymptotic $\rightarrow$ for measuring speed (O, Ω, Θ)
- 2) mathematical $\rightarrow$ for writing logic ($\Sigma, \in, \forall$)
- 3) Pseudocode $\rightarrow$ for writing algorithms
- 4) Flow Chart $\rightarrow$ for visual representation

why learn $\rightarrow$ To compare which algorithm is faster

Asymptotic notations =>

i) Asymptotic notation is a mathematical way to describe the performance of an algorithm.
ii) It shows how time or space grows when input size n becomes very large
iii) It ignores constants and small terms
iv) It is used in time complexity and space complexity analysis

~~Sem~~ Why Asymptotic notation used => same as notation

Types =>

1) Big-O Notation (O) =>
i) Represents the upper bound ii) shows the worst-case performance
iii) formula : $f(n) \leq c \cdot g(n)$
iv) when we want to guarantee " won't take more than this"

2) Big-Omega Notation (Ω) =>
i) Represents the lower bound ii) shows the best-case performance.
iii) formula : $f(n) \geq c \cdot g(n)$
iv) when we want " at least this much time

3) Big-Theta Notation (Θ) =>
i) Represents tight bound ii) shows both upper & lower bounds
iii) formula : $c_1 g(n) \leq f(n) \leq c_2 \cdot g(n)$
iv) when we know exact growth rate.

Divide and conquer =>

i) Divide and conquer is an algorithm design technique
ii) It solve a problem by dividing it into smaller sub-problems
iii) Each sub-problem is solved recursively
iv) Solutions are then combined to get the final result

method =>

1) Divide => i) Break the problem into smaller sub-problems
ii) Sub-problems are of same type as original problem

2) conquer => i) solve each sub-problem recursively
ii) If sub-problem is small enough, solve it directly

3) combine => i) combine the result of sub problems
ii) Get the final solution.

General Recurrence Relation => $T(n) = aT(n/b) + f(n)$

Binary search => i) A searching Algorithm ii) works on a sorted array
iii) Repeatedly divides search space into two halves.

Binary search Algorithm =>

BinarySearch (A, low, high, key)
1) If low > high, return -1
2) mid = (low + high) / 2
3) If A[mid] == key, return mid
4) If key < A[mid], search left half
5) Else search Right half

Time complexity of Binary search

$T(n) = T(n/2) + 1$

$T(n) = O(\log n)$

space complexity =>

i) Recursive version : $O(\log n)$
ii) Iterative version : $O(1)$

P and NP classes ⇒ Relation of P and NP ⇒ $P \subseteq NP$, $P = NP$ or $P \neq NP$

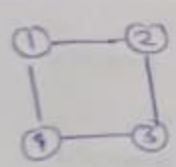
Feature	P	NP
full form	Polynomial time	Non deterministic Polynomial time
Solution	Easy to find	Hard to find
verification	Easy	Easy
machine	Deterministic TM	Non deterministic TM (Turing machine)
Efficiency	Efficient	Possibly inefficient
Eg	Sorting	TSP, SAT

NP-hard v/s NP complete ⇒ Relation $NP\text{-complete} \subseteq NP\ Hard$

NP Hard	NP complete
1) may not belong to NP	Always belongs to NP
2) Solution verification not required to be polynomial	solution verification is polynomial time
3) Can be decision, optimization, or undecidable	only decision problems
4) At least as hard as NP problems	Hardest problems within NP
5) Polynomial-time solution does not ~~empty~~ empty P = NP	· Polynomial-time solution implies P = NP
6) Superset category	subset of NP-Hard
7) Eg ⇒ TSP (optimization), Halting Problem	Example : SAT, vertex cover.

Hamiltonian cycle ⇒ i) Each vertex is visited once
ii) Start and end vertex are same, iii) graph must be connected
iv) Different from Euler cycle

Space = O(n)

Eg ⇒ 

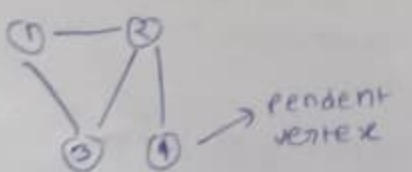
⇒ 1 2 3 4 1 ✓ Hamiltonian graph
1 4 3 2 1 ✓
2 3 4 1 2 ✓

Time ⇒ $T(n) = (n-1) \cdot T(n-1)$

$O(n!)$

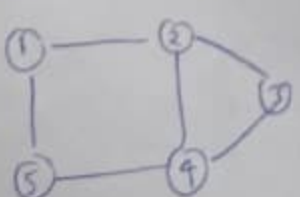
ii) 

⇒ 1 2 3 4 5 ✗ Articulation point exist so the
2 1 3 4 5 ✗ graph is not Hamiltonian

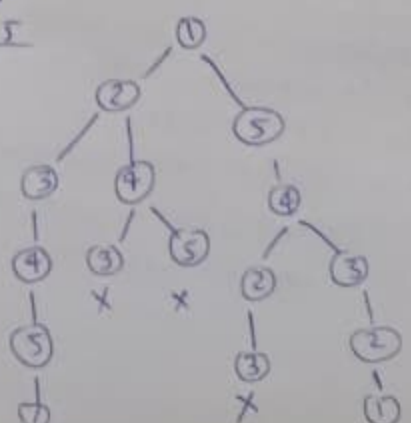
iii) 

No Hamiltonian cycle exist

How to solve using backtracking ⇒ apply using ⇒ DFS ⇒ NP-complete problem

Eg ⇒ 

⇒ 



Solutions ⇒ 1 2 3 4 5 1
1 5 4 3 2 1

Linear speed up theorm =>

1) Definition => i) Increasing computational Resources (CPU speed, number of processors, or memory) can proportionally reduce the execution time of an algorithm.

ii) Ideally, doubling the resources halves the execution time

2) Speedup formula => $S = \frac{T_1}{T_p}$

T_1 = execution time on a single processor

T_p = execution time on p processors

S = speedup

3) Linear speedup condition =>

i) Achieved when :-> $S \simeq P$

i.e. speedup is equal to the number of processors

ii) Each can additional processor gives proportional improvement in execution time

4) Assumptions => i) No significant overhead in communication or synchronization

ii) Workload can be perfectly divided among processors.

iii) Algorithm has parellelizable tasks.

5) Limitation => i) overhead of inter-processor communication

ii) Synchronization delays

iii) Non-parallelizable portions of the program

6) eg => A task takes 10 hours on a single processor

using 5 processors, task completes in $\simeq$ 2 hours

speedup => $S = \frac{10}{2} = 5$